

RAG Reranking and Retrieval Evaluation

From Top- K Retrieval to Context Precision and Context Recall

Detailed Notes - Agentic AI Bootcamp

Contents

1	Why Reranking Exists	3
2	The Two-Stage Retrieval Architecture	3
3	Why Vector Similarity Alone Is Not Enough	4
4	What Does Cohere Use for Reranking?	5
4.1	Query and Document Are Considered Together	6
5	Is Cohere Rerank an LLM?	6
6	How Cohere Rerank Fits Into a RAG Pipeline	7
7	Why Not Simply Increase Top-K?	8
8	Reranking and Context Precision	9
8.1	Context Precision Formula	10
9	Context Recall	10
9.1	Why Context Recall Needs a Reference	10
10	Context Precision vs. Context Recall	11
11	A Simple Example	11
12	Putting the Metrics and Reranker Together	12
13	Typical Failure Patterns	12
13.1	High Recall, Low Precision	12
13.2	Low Recall, High Precision	13
13.3	Low Recall, Low Precision	13
14	An Important Limitation of Reranking	14
15	Reranking Is Not the Same as Increasing Top-K	14
16	Reranking and Token Budget	14

17 Where Reranking Sits in the Bootcamp	15
18 A Practical Mental Model	15
19 Key Takeaways	16
20 One-Sentence Summary	16
21 References and Further Reading	16

1 Why Reranking Exists

A Retrieval-Augmented Generation (RAG) system normally has three broad stages:

1. **Retrieve candidates** from a large document collection.
2. **Rerank the candidates** according to their relevance to the user's query.
3. **Generate an answer** using the best retrieved context.

A useful mental model is:

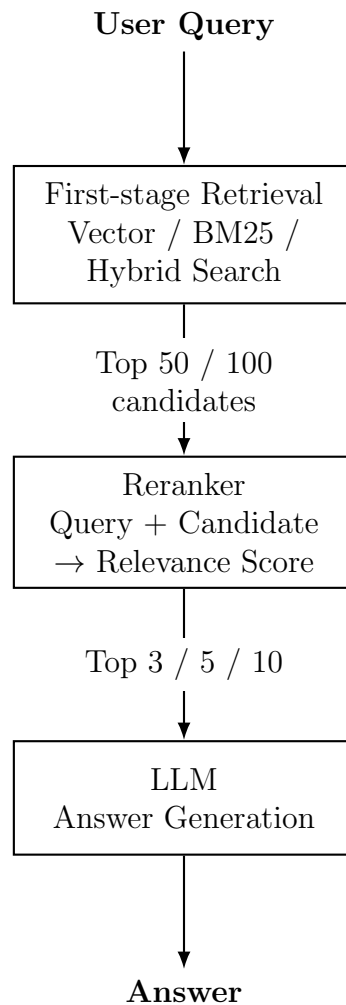
Retriever = candidate generator
Reranker = relevance judge
Generator = answer synthesizer

The key reason for separating these stages is that retrieval and ranking optimize different objectives.

- The first-stage retriever should emphasize **recall**: do not miss potentially useful information.
- The reranker should emphasize **precision**: among the candidates, put the most useful information first.
- The generator should use the final context to produce a grounded answer.

2 The Two-Stage Retrieval Architecture

A typical RAG pipeline can look like this:



For example, a collection may contain 10,000 chunks. A vector database might retrieve the best 50 candidates. A reranker can then examine those 50 candidates more carefully and select the best 5 for the final LLM prompt.

This is generally preferable to asking the vector database to directly return only five chunks, because the first stage can deliberately optimize for **recall** while the second stage restores **precision**.

3 Why Vector Similarity Alone Is Not Enough

Embedding retrieval answers a question approximately like:

“Which documents are semantically similar to this query?”

That is useful, but semantic similarity is not identical to answer relevance. Consider the query:

What is the company’s policy for employees working remotely from another country?

A first-stage retriever might return:

1. Working From Abroad

2. Remote Work Policy
3. Employee Travel Policy
4. Office Attendance Policy
5. International Payroll

All five can be semantically related to the query. However, only some may contain the precise information required to answer it.

A reranker can compare the query directly against each candidate and assign a relevance score:

Candidate	Example relevance score
Working From Abroad	0.91
Remote Work Policy	0.84
Employee Travel Policy	0.54
International Payroll	0.37
Office Attendance Policy	0.29

The exact numerical scores are illustrative. The important idea is that the reranker produces a **query-dependent ordering**.

4 What Does Cohere Use for Reranking?

Cohere provides dedicated **Rerank models**. These are neural relevance models designed specifically for ranking documents with respect to a query.

The current Cohere documentation lists, among others:

- rerank-v4.0-pro
- rerank-v4.0-fast
- rerank-v3.5

Cohere describes Rerank as a semantic ranking model: given a query and a list of documents, it predicts relevance and returns the documents ordered from most to least relevant.

The important architectural point is:

Cohere Rerank is model-based, but it is not same as a general-purpose LLM (i.e. it does not “have generative capabilities”).

It is better described as a **specialized transformer-based relevance/reranking model**, which is a trained Neural-Network.

4.1 Query and Document Are Considered Together

The key distinction from embedding retrieval is that the reranker directly considers the query and candidate document together.

Conceptually:

$$(\text{Query}, \text{Document}) \longrightarrow \text{Relevance Score}$$

Cohere describes this as applying **cross-attention** for fine-grained ranking.

By contrast, a typical embedding retrieval flow is:

$$\text{Query} \longrightarrow \mathbf{q}$$

$$\text{Document} \longrightarrow \mathbf{d}$$

followed by a similarity calculation such as:

$$\text{sim}(\mathbf{q}, \mathbf{d})$$

The reranker therefore performs a more direct query-document interaction.

5 Is Cohere Rerank an LLM?

Well, not precisely! LLM is a **Large Language Model**, while Reranker is usually a Transformer based model, which is a trained **Neural Network** to predict scores based on the relevancy of the chunks with respect to the user *query*.

Thus,

Reranker = transformer-based relevance model
Generator = general-purpose generative LLM

Component	Primary job
Embedding model	Converts query/documents into vectors.
Vector database / search engine	Retrieves candidate documents efficiently.
Reranker	Scores candidate documents for query relevance and reorders them.
Generative LLM	Uses the selected context to generate the final response.

Yes, there are several popular implementations and models specifically designed or commonly used for LLM-based reranking:

1. **RankGPT:** A widely used methodology that leverages frontier models (like GPT-4) to perform listwise ranking of retrieved passages. Orchestration frameworks like `LlamaIndex` offer built-in modules (like `RankGPTReranker` or `LLMRerank`) that implement this out-of-the-box.

2. **RankZephyr:** An open-weight model specifically distilled to replicate the ranking capabilities of larger models. This allows developers to run a highly capable LLM reranker locally without the heavy API costs of a frontier model.
3. **Qwen3 Reranker:** A family of models that use a generative LLM architecture to score document relevance based on the probability of generating a "yes" versus "no" for a query-document pair.

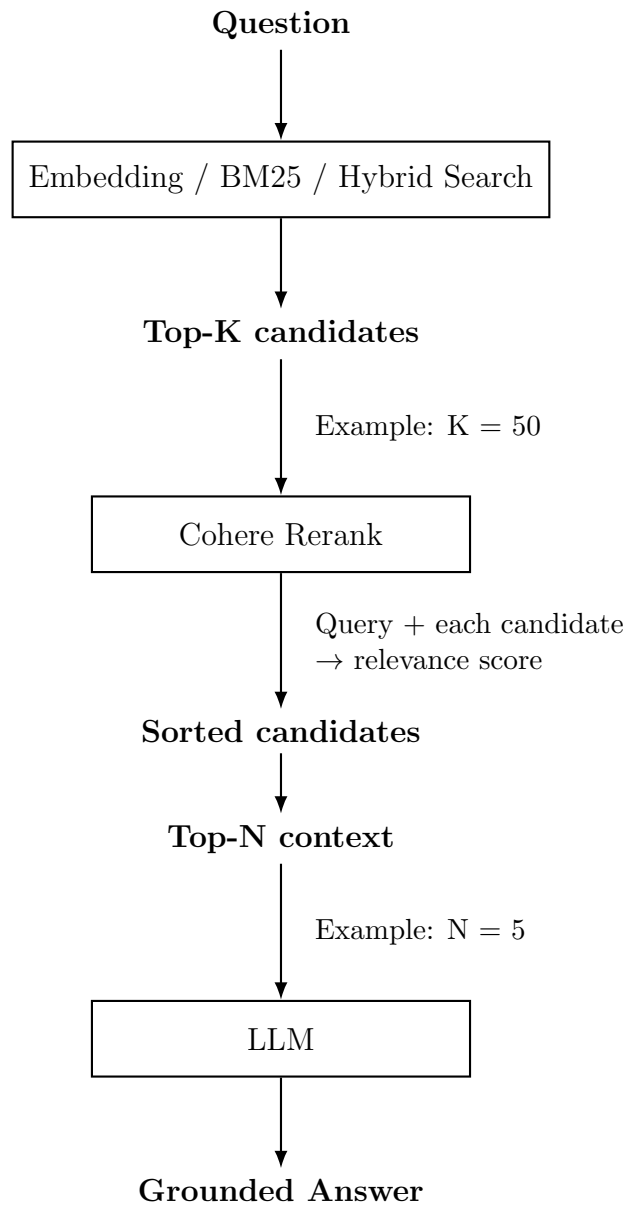
Consideration	Traditional Cross-Encoders	LLM Rerankers
Latency	Usually adds tens to low-hundreds of milliseconds per query.	Can take seconds, depending on the model size and candidate pool.
Cost	Very cheap to run, easily hosted on moderate hardware.	Can become prohibitively expensive due to high token usage per query.
Out-of-Domain Accuracy	Can struggle if the domain vocabulary is vastly different from the training data.	Excellent zero-shot performance across diverse or niche topics.

Table 2: The Trade-offs between Traditional Cross-Encoders and LLM Rerankers

Cohere uses dedicated neural reranking models that directly evaluate query-document relevance.

6 How Cohere Rerank Fits Into a RAG Pipeline

The practical flow is:



The values $K = 50$ and $N = 5$ are examples, not universal recommendations. The important design principle is:

$$K_{\text{retrieval}} > N_{\text{final context}}$$

The first stage casts a wide net. The reranker then narrows that set.

7 Why Not Simply Increase Top- K ?

Suppose the retriever initially returns:

$$K = 5$$

and the correct chunk is ranked at position 8.
 The correct chunk is never seen by the generator.
 Increasing K to 20 may recover the correct chunk:

$$K = 20$$

but now the LLM may receive many irrelevant chunks.
This creates a trade-off:

	Advantage	Risk
Small K	Less context, lower token cost	May miss relevant information
Large K	Better chance of retrieving relevant information	More irrelevant context and higher token usage
Reranking	Recovers candidates first, then improves ordering	Adds computation/latency

This is why reranking is especially useful when retrieval quality is important and the final LLM context budget is limited.

8 Reranking and Context Precision

Reranking is closely connected to the RAG evaluation metric called **Context Precision**.

Context Precision evaluates whether relevant chunks are ranked above irrelevant chunks in the retrieved context.

The central question is:

Are the useful chunks near the top of the retrieval results?

This is different from simply asking whether a useful chunk exists somewhere in the top- K .

For example:

Rank	Chunk	Relevant?
1	A	Yes
2	B	Yes
3	C	No
4	D	No
5	E	Yes

This is generally better than:

Rank	Chunk	Relevant?
1	C	No
2	D	No
3	E	Yes
4	A	Yes
5	B	Yes

Both retrieve the same number of relevant chunks. The second ordering is worse because irrelevant material occupies the highest ranks.

8.1 Context Precision Formula

Ragas describes Context Precision in terms of precision at each rank:

$$\text{Precision@k} = \frac{\text{TruePositives@k}}{\text{TruePositives@k} + \text{FalsePositives@k}}$$

The Context Precision metric aggregates these rank-sensitive precision values over the retrieved context.

Conceptually:

Context Precision $\approx$ How strongly relevant chunks are concentrated near the top

Higher is better.

9 Context Recall

Context Recall asks a different question:

Did the retrieval process find the information needed to answer the question?

It is therefore concerned with **missing information**.

For example, suppose the reference answer requires three pieces of information:

1. Policy applies to remote employees.
2. Manager approval is required.
3. Core working hours must be maintained.

If the retrieved context supports only two of these three claims, the context has incomplete recall.

For the LLM-based Ragas formulation:

$$\text{Context Recall} = \frac{\text{reference claims supported by retrieved context}}{\text{total reference claims}}$$

Thus, in the example:

$$\text{Context Recall} = \frac{2}{3} \approx 0.67$$

Higher is better.

9.1 Why Context Recall Needs a Reference

Recall is fundamentally about what was **missed**. To know what was missed, we need some representation of what should have been available.

Ragas can use a reference answer as a proxy for the reference context. It breaks the reference into claims and checks whether those claims can be attributed to the retrieved context.

There are also non-LLM and ID-based variants when explicit reference contexts or identifiers are available.

10 Context Precision vs. Context Recall

The distinction is critical:

	Context Precision	Context Recall
Main question	Did we rank useful chunks high?	Did we retrieve the required information?
Primary concern	Irrelevant context / bad ordering	Missing relevant information
Focus	Precision of retrieved ranking	Completeness of retrieval
Reference	Can use reference answer/-context	Requires a reference or ground truth representation
Higher score	Better	Better
Typical remedy	Better ranking, reranking, retrieval tuning	Larger candidate pool, better embeddings, hybrid search, better chunking

Thus,

Precision: “Did I retrieve the right things near the top?”

Recall: “Did I retrieve everything important?”

11 A Simple Example

Suppose the correct answer requires three chunks:

$$R = \{A, B, C\}$$

The retriever returns:

$$[A, D, E, B, F, C]$$

All three relevant chunks were retrieved.

Therefore, context recall is high:

$$\text{Recall} = \frac{3}{3} = 1.0$$

But the relevant chunks are not all near the top.

If a reranker produces:

$$[A, B, C, D, E, F]$$

the set of retrieved information has not changed, but the ordering has improved substantially.

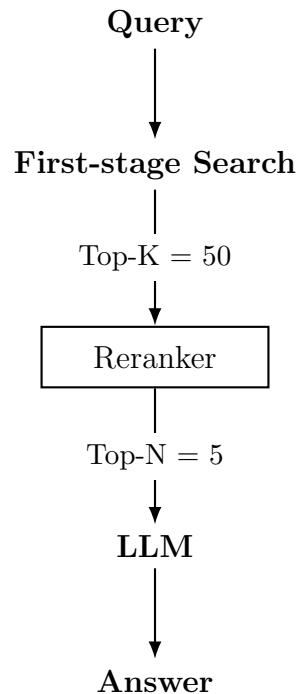
This illustrates an important point:

Reranking can improve retrieval quality without discovering new documents.

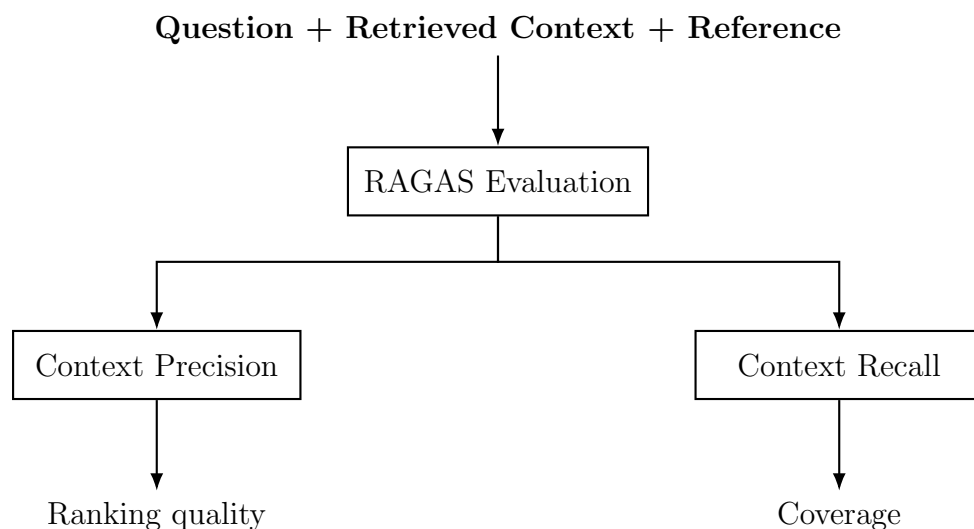
The first-stage retriever determines the candidate set. The reranker improves the ordering of that candidate set.

12 Putting the Metrics and Reranker Together

A useful conceptual pipeline is:



During evaluation:



The metrics help diagnose different failure modes.

13 Typical Failure Patterns

13.1 High Recall, Low Precision

The system retrieved the required information, but buried it under irrelevant material.

Example:

$$\text{Recall} = 0.95, \quad \text{Precision} = 0.45$$

Possible remedies:

- Add a reranker.
- Improve first-stage ranking.
- Improve query formulation.
- Remove noisy or duplicate chunks.
- Improve metadata filtering.

13.2 Low Recall, High Precision

The chunks that were retrieved are useful, but the system failed to retrieve all the information required.

Example:

$$\text{Recall} = 0.45, \quad \text{Precision} = 0.90$$

Possible remedies:

- Increase the first-stage retrieval K .
- Improve embeddings.
- Consider hybrid search.
- Improve chunking.
- Improve query expansion or rewriting.
- Review metadata filters that may be excluding useful documents.

13.3 Low Recall, Low Precision

The retrieval system has a broader problem.

Possible causes include:

- Poor chunking.
- Poor embeddings.
- Incorrect metadata.
- Weak query formulation.
- Inappropriate vector distance/search configuration.
- Poor source-document extraction.

Reranking alone will not solve this if the correct information never reaches the reranker.

14 An Important Limitation of Reranking

Reranking is not magic.

Suppose the correct chunk is not among the first-stage candidates:

Correct chunk $\notin$ Top- K

Then:

Reranker cannot rank it

because it never received it.

This gives the fundamental relationship:

Reranking improves ordering within the candidate set; it does not replace candidate generation.

Therefore, a strong RAG architecture must optimize both:

Candidate Recall + Candidate Ranking

15 Reranking Is Not the Same as Increasing Top- K

These two techniques solve different problems.

Technique	What it changes	Primary effect
Increase Top- K	Expands the candidate set	Potentially improves recall
Reranking	Reorders the candidate set	Improves precision/order
Improve embeddings	Changes semantic representation	Can improve candidate retrieval
Hybrid search	Combines lexical and semantic signals	Can improve recall for exact and semantic matches
Better chunking	Changes retrieval units	Can improve both recall and precision

In production RAG systems, these techniques are often combined.

16 Reranking and Token Budget

A practical reason for reranking is context-budget management.

Suppose:

$$K = 50$$

retrieved candidates contain approximately 40,000 tokens in total.

It may be impossible or undesirable to send all 50 candidates to the generator.

A reranker can select:

$$N = 5$$

high-value chunks, producing a much smaller final context.
 The conceptual optimization is:

High candidate recall $\longrightarrow$ High relevance $\longrightarrow$ Small final context

This can reduce:

- Input token usage.
- Latency.
- Context noise.
- The chance that the LLM focuses on irrelevant information.

Cohere’s current Rerank documentation also supports reranking large candidate sets, subject to endpoint and model limits.

17 Where Reranking Sits in the Bootcamp

The bootcamp’s RAG sequence is deliberately structured around this progression:

1. **Embeddings** — understand semantic representation.
2. **Vector database** — understand candidate retrieval.
3. **Top- K** — understand the recall/context trade-off.
4. **RAG pipeline** — connect retrieval to generation.
5. **Context Precision and Context Recall** — measure retrieval quality.
6. **Reranking** — introduce a second-stage relevance model.

This naturally leads to the broader RAGOps lesson:

Do not tune a RAG system by changing Top- K blindly. Measure what retrieval is doing.

The bootcamp plan explicitly identifies Context Precision and Context Recall as RAG evaluation topics, and the later continuous-evaluation material uses evaluation metrics as a quality gate for AI-system changes.

18 A Practical Mental Model

Below is a layman’s way to understand the contrast between the two:

Question	System component
Where might the answer be?	First-stage retriever
Which retrieved chunk is most relevant?	Reranker
Did we retrieve enough of the required information?	Context Recall
Did we rank useful chunks above noisy chunks?	Context Precision
How do we turn the context into an answer?	Generative LLM

19 Key Takeaways

1. **Reranking is a second-stage retrieval technique.**
2. Cohere provides dedicated **Rerank models** rather than requiring a general-purpose chat LLM to rank documents.
3. Cohere Rerank directly considers the **query and candidate document together** and produces relevance scores.
4. The first-stage retriever should generally be optimized for **recall**.
5. The reranker should improve **precision and ordering** within the retrieved candidate set.
6. Increasing Top- K can improve the chance of finding the right information, but it also increases noise.
7. Reranking cannot recover a document that the first-stage retriever never returned.
8. **Context Precision** measures whether relevant retrieved chunks are ranked highly.
9. **Context Recall** measures whether the retrieved context contains the information needed to support the reference answer.
10. Good RAG systems therefore need both **candidate recall** and **high-quality ranking**.

20 One-Sentence Summary

Retrieve broadly, rerank precisely, pass only the strongest context to the LLM, and measure retrieval quality using Context Precision and Context Recall.

21 References and Further Reading

1. Cohere, “Cohere’s Rerank Model (Details and Application).”
<https://docs.cohere.com/v2/docs/rerank>
2. Cohere, “Best Practices for using Rerank.”
<https://docs.cohere.com/docs/reranking-best-practices>
3. Cohere, “Master Reranking with Cohere Models.”
<https://docs.cohere.com/v2/docs/reranking-with-cohere>
4. Ragas Documentation, “Context Precision.”
https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_precision/

5. Ragas Documentation, “Context Recall.”
https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_recall/